

Credit Card Fraud Detection – Technical Report

Project Overview

In a world where financial fraud is lurking in the shadows, our mission was clear: build a system that can spot suspicious transactions before they burn a hole in someone's wallet. This project is all about detecting fraudulent credit card activity using machine learning – and doing it in style.

We used the popular **Kaggle Credit Card Fraud Detection dataset**, which contains real, anonymized transactions made by European cardholders. What makes this dataset spicy? Only **0.17%** of the transactions are actually frauds. That's right – finding fraud here is like looking for a needle in a haystack... that's on fire... and the needle is invisible. So yes, it's challenging.

Our objective was to:

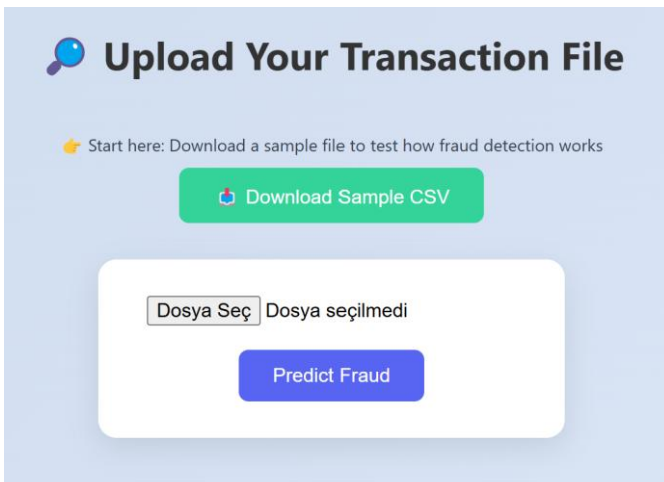
- Understand the dataset's structure and challenges
- Handle the massive class imbalance
- Build a model that can accurately classify fraud vs. safe transactions
- Deploy that model into a beautiful, user-friendly web app

The outcome? A fully functional, cleanly deployed fraud detector where users can upload a CSV file and instantly find out whether their transactions look shady or safe.

This project combines **exploratory data analysis**, **feature scaling**, **class imbalance handling (hello SMOTE!)**, and **model optimization** – all topped off with a modern **Flask-based web interface** that even your grandma could use.

If you're ready to peek under the hood, let's dive in.

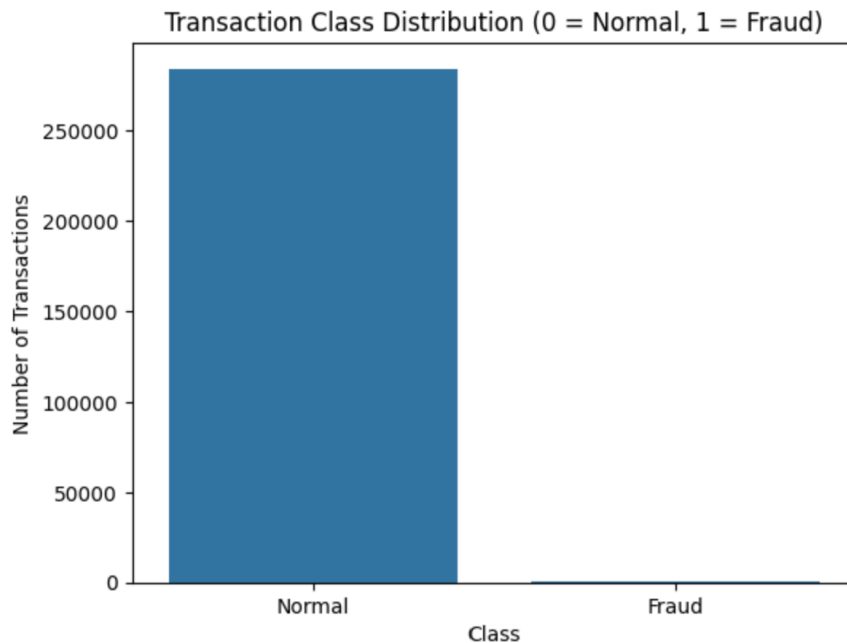
Final app interface:



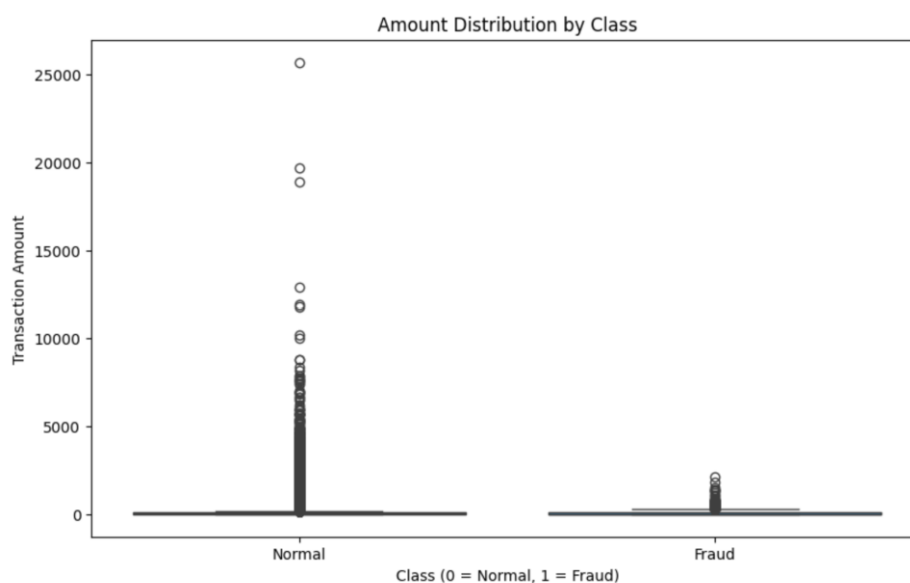
🔍 Exploratory Data Analysis (EDA)

Before you try to fight fraud, you need to understand what it looks like. That's where EDA comes in – it's like putting on your detective hat and looking for patterns, red flags, and shady behavior in the dataset.

First stop: class distribution. As mentioned earlier, fraud is extremely rare. A quick count plot reveals that fraud cases are almost invisible next to the avalanche of safe transactions. Not surprising, but important to keep in mind when building models.

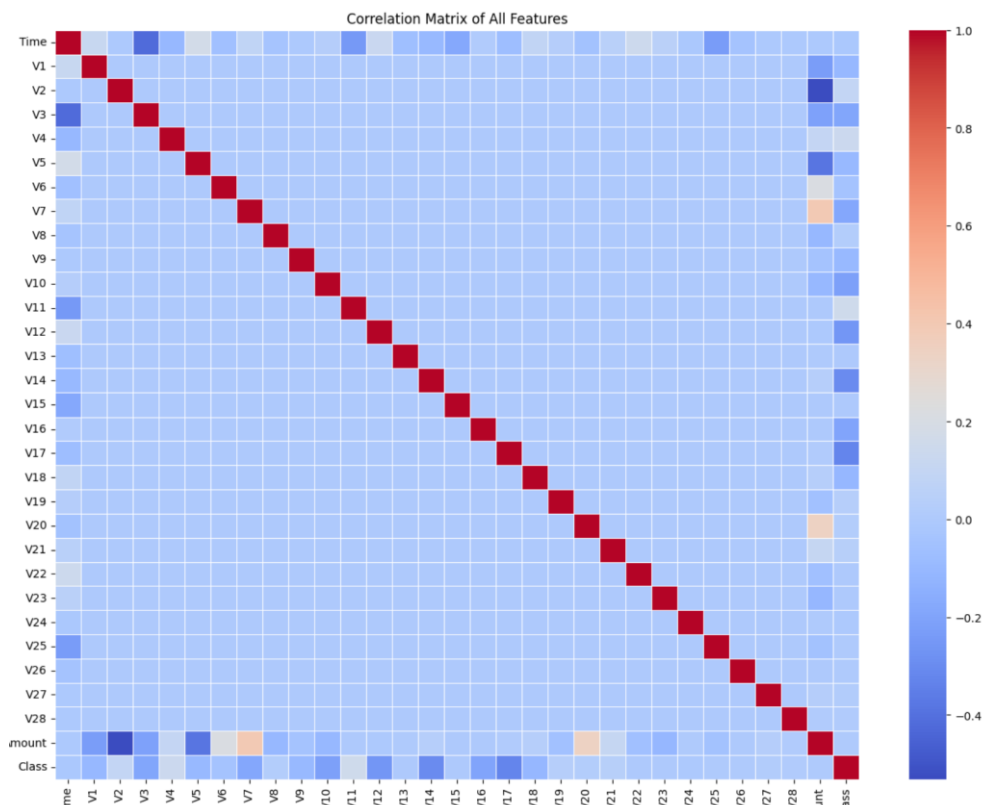
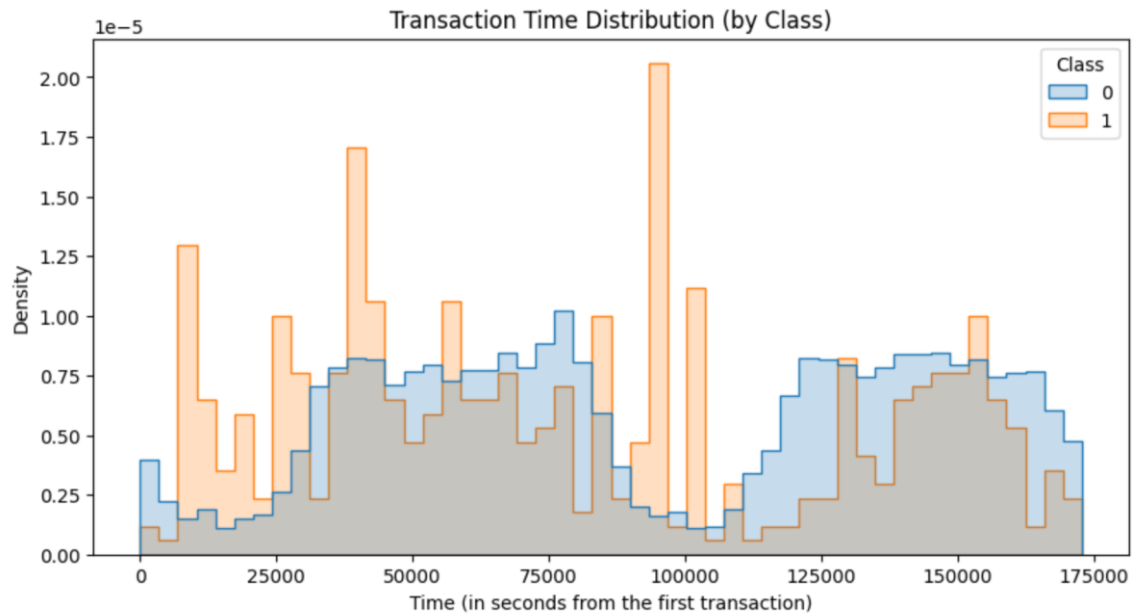


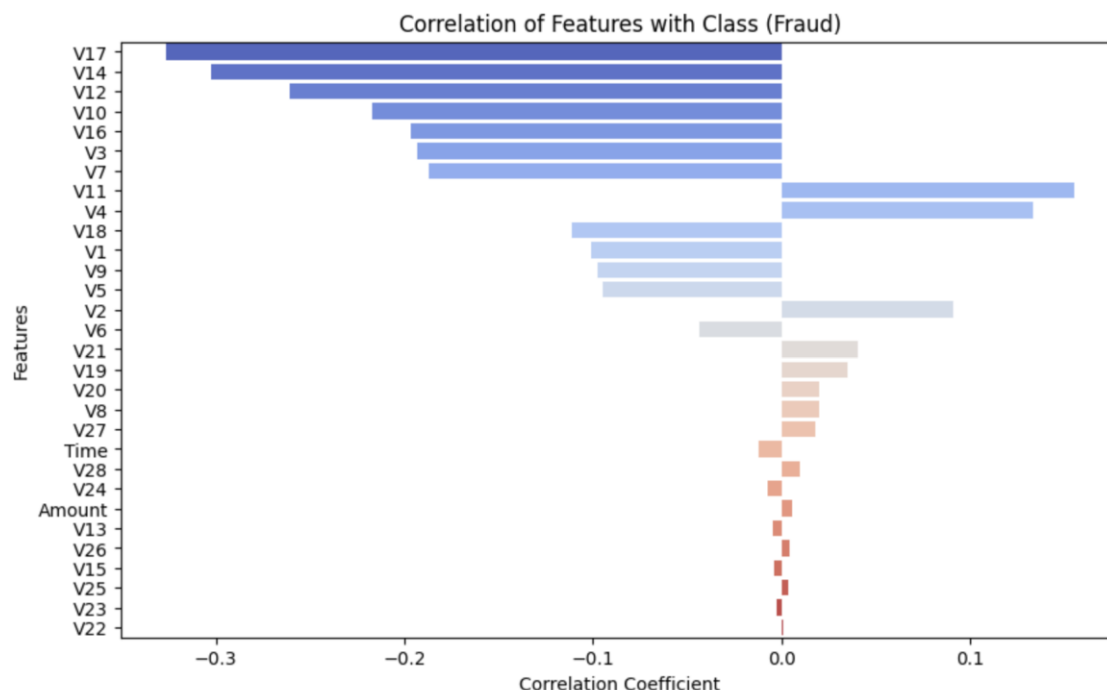
Next up, we wanted to see if fraud transactions look different in terms of amount. Using boxplots, we visualized the distribution of transaction amounts across fraud vs. normal classes. Result? Fraud transactions tend to be smaller on average – probably to avoid detection. Sneaky!



We also checked for correlations among the anonymized features (V1-V28). There weren't any obvious multicollinearity issues, but certain components stood out as more fraud-informative than others. We took note of those for later feature importance analysis.

Finally, we ran some basic statistical summaries and scatter plots just to get a feel for the data landscape. Even with anonymized features, data has a story – and EDA is how you start reading it.





🔧 Preprocessing & Class Imbalance Handling

Now that we had a solid understanding of the battlefield, it was time to gear up for modeling. And in fraud detection, the biggest beast to tame is **class imbalance**.

Out of nearly 285,000 transactions, only around 490 were fraudulent. That means any model that simply predicts "safe" for everything would still score over 99% accuracy. Pretty useless, right?

To fix this, we used **SMOTE (Synthetic Minority Over-sampling Technique)**. SMOTE creates synthetic fraud examples by interpolating between real fraud samples, giving the minority class more representation without duplication. This helps models actually learn what fraud *looks* like, instead of ignoring it.

Next, we scaled the features. Since most of the data was already PCA-transformed (hello, V1-V28), we didn't need any heavy-duty encoding. But the 'Amount' and 'Time' columns needed some TLC. We used **MinMaxScaler** to bring them into a similar range so that distance-based models wouldn't get confused by magnitude differences.

We then split the data into **train/test sets**, keeping a healthy 80/20 split. After SMOTE was applied *only* on the training set (never the test!), we were ready to model.

Modeling: From Baseline to Beast Mode

Alright, it was time to throw some algorithms at our fraud problem and see who survives the gauntlet. We didn't go straight for the flashy stuff – first, we laid a solid foundation.

1. **Logistic Regression (Baseline):** We started with a no-frills logistic regression using the raw, imbalanced dataset. No resampling, no adjustments. The point? Get a benchmark. Accuracy was deceptively high (because of the class imbalance), but recall for the fraud class was... let's say humble. And that's okay – it was never meant to be a hero, just the starting line.
2. **SMOTE + Logistic Regression:** Next up, we introduced SMOTE to the logistic regression model. With the classes now more balanced, the model had a fairer shot at catching frauds. And lo and behold, performance improved. Precision, recall, and F1-score got a nice boost. Still not groundbreaking, but now we were on the map.
3. **XGBoost Classifier:** Time to bring out the big guns. XGBoost is like the special forces of classification. For this model, instead of using SMOTE, we handled class imbalance natively using the **scale_pos_weight** parameter. This tells XGBoost to pay more attention to the underrepresented fraud class, without touching the data itself.

We also tuned hyperparameters by manually (for XGBoost). The reason why we did this process manually is; we wanted to see how each parameter affects the model performance (That's why we didn't use RandomizedSearchCV). Tweaking stuff like:

- learning_rate
- max_depth
- n_estimators

The result? A highly performant model that crushed it on both training and test sets, showing solid F1-score and precision without overfitting.

Model Comparison Summary

Now let's take a step back and compare our three models side by side. The goal here isn't just to chase accuracy – we're focused on metrics that matter when detecting rare events: **recall**, **precision**, and **F1-score** for the fraud class.

Final Comparison: All Models

Model	Precision	Recall	F1 Score	False Positives	Missed Frauds
Baseline Logistic	83.33%	66.33%	73.86%	13	33
SMOTE Logistic	5.77%	91.84%	10.86%	1469	8
XGBoost	88.17%	83.67%	85.86%	11	16

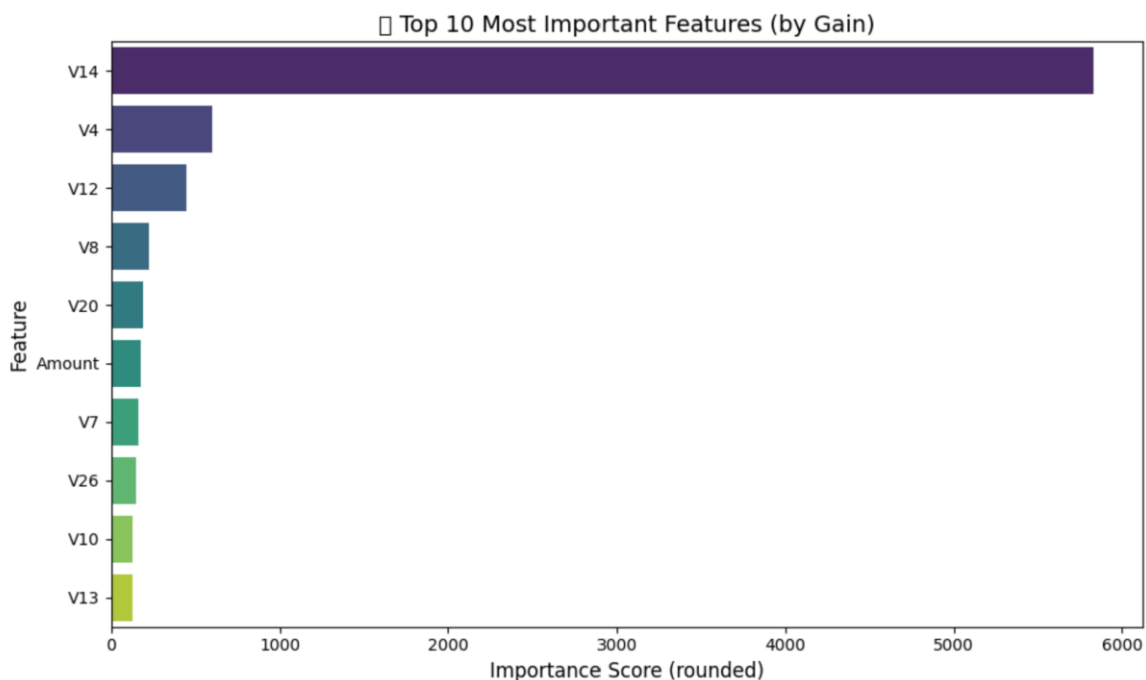
✅ The baseline model was just that – a baseline. ✅ SMOTE helped give logistic regression a fairer shot. ✅ XGBoost, especially with `scale_pos_weight`, was the clear winner in performance and robustness.

📌 Feature Importance

So which features actually helped us catch the bad guys? That's where **feature importance** comes in. Thanks to XGBoost, we got a built-in way to assess how much each feature contributed to the final predictions.

Despite the feature names being anonymized (V1-V28), we can still interpret their impact. Some components had a much higher influence in determining fraud than others.

We used XGBoost's `plot_importance()` function to visualize the most valuable features. Not surprisingly, features like V14, V4, and V12 rose to the top.



📌 Why We Use F1-Score (And Not Accuracy)

Let's clear the air for anyone thinking: "I can get 99.8% accuracy by just predicting everything as safe!"

And... you wouldn't be wrong. But you also wouldn't be useful.

In highly imbalanced datasets like this one, accuracy is a deceptive metric. A model that labels every transaction as "not fraud" will still get 99.8% accuracy – because the vast majority are indeed safe. But it would completely miss the tiny sliver of actual frauds.

That's why we use **F1-score**, which balances **precision** (how many predicted frauds are actually fraud) and **recall** (how many actual frauds were caught). It's the best metric when your positive class is rare and important – exactly our situation.

Model Deployment

Once our fraud-hunting model was trained, tuned, and ready to roll, it was time to put it into action. A machine learning model isn't worth much unless people can actually use it — so we wrapped it in a smooth, minimalistic **Flask web application**.

Backend

- We used pickle to save the trained XGBoost model.
- The model was then loaded into a Flask API that accepts CSV uploads, processes the data, and returns predictions.
- Pandas handles file reading and transformation on the fly.

Frontend

- We crafted a responsive, elegant HTML interface with CSS animations and hover effects.
- Users can upload their own transaction file, or download a sample CSV to try it out first.
- After uploading, a summary box appears indicating how many transactions are safe vs fraudulent.

Deployment

- We hosted the entire app on **Render**, which made deployment smooth and efficient.
- Render watches our GitHub repo for updates and redeploys automatically — continuous integration made easy.

This deployment brings all our hard work together into an interface that's not only functional but actually fun to use. Whether you're a data scientist or just someone trying to check a CSV for fraud, you're covered.

See the picture below:



Upload Your Transaction File

👉 Start here: Download a sample file to test how fraud detection works



Download Sample CSV

Dosya Seç

Dosya seçilmedi

Predict Fraud



49 safe transactions detected. 🚨

3 fraudulent transactions detected.

✅ Conclusion

From raw data to a deployed web application, this project has been a full-stack machine learning adventure — with a purpose. We tackled a highly imbalanced and high-stakes problem, where even a single missed fraud could mean thousands of dollars lost.

We experimented with multiple models, battled the imbalance beast with SMOTE and XGBoost tricks, and wrapped the whole thing in a polished web interface. But beyond the code and charts, this project shows what machine learning is really about: solving real problems with clarity, creativity, and care.

Whether you're a fraud analyst, a curious data geek, or someone who stumbled upon this app while sipping coffee — our goal was to make something useful, understandable, and a little bit cool. Mission accomplished.

This project was handcrafted with 🧠 curiosity, ☕ caffeine, and 🐛 fixing dozens of meaningless bugs.